

Why Compilers Rewrite Your Arithmetic

Lecture Notes — Week 4: Conversion, Hanoi, and Recursion with the Runtime Stack

Auqib Hamid Lone

Department of Computer Science & Engineering
Government College of Engineering & Technology (GCET), Ganderbal

September 9, 2026

4.1 Why This Week Exists: The Missing Rewrite

Read alongside: Horowitz & Sahni, *Fundamentals of Data Structures*, Ch. 3 (chapter-level); Aho, Hopcroft & Ullman, *Data Structures and Algorithms*, Ch. 2 (chapter-level).

Four questions organize this chapter, stated here exactly as the deck states them: why is $a+b*c$ hard to evaluate as written; how does one stack turn infix into postfix or prefix; how does one stack turn prefix back into postfix; and why does moving 64 disks take longer than the age of the universe. Compressed to a plan: name the three notations, convert between them with a stack, then let functions call themselves, and price what that costs.

The handoff from Week 3 is exact. Week 3 gave us the Stack ADT (`push/pop/peek`, `top` with $-1 = \text{empty}$, each $O(1)$ worst case) over an array. It applied one stack to two jobs: checking brackets (Lab P1) and *evaluating* a postfix string *already in postfix* (Lab P2). What it never did: produce that postfix string from what humans actually write. This week’s job takes $a+b*(c-d)/e$ as typed, rewrites it for the stack, then studies the stack that rewrites function calls: recursion.

Figure 4.1 places the work in the course’s expression processor. Weeks 1–3 built stages 1, 2, and 4; the missing stage is the rewrite in the middle. Recursion, the second half of the chapter, explains how stage 4’s evaluator and every function call actually run. One program learning new tricks each week — the course thread.



Figure 4.1: The calculator pipeline: tokenize, check brackets (Week 3), convert (this week), evaluate (Week 3).

Two arc notes close the framing. Week 2 priced things: name the operation, give the bound, state the case — and every claim in this chapter repeats that shape. Week 3’s close promised “from postfix to prefix and back, Hanoi, and recursion with the runtime stack,” and this chapter pays exactly that promise. Week 5 (queues) then reuses the same conversion discipline for buffering, and Week 11 returns with expression trees. The arc again: state the contract, implement it once, analyse what it costs.

4.2 Three Ways to Write the Same Arithmetic

Read alongside: Horowitz & Sahni, Ch. 3 (chapter-level) — the standard treatment of the three notations.

4.2.1 Why infix is hard to evaluate

Infix mixes three rules, and a single left-to-right scan cannot apply all three without remembering. First, in $a+b*c$, the $+$ comes first but runs second — position lies about order, because $*$ outranks $+$. Second, in $a-b-c$, the same symbol twice needs a tie-break: left to right gives $(a-b)-c$, while right to left would give $a-(b-c)$, and only a stated tie-break picks one. Third, in $(a+b)*c$, brackets override everything — the scanner must remember how deep it is. In one line: infix mixes precedence, associativity, and parentheses, so evaluation needs memory for what is pending.

Definition (Infix, postfix, prefix). *Infix* puts the operator between its operands ($a+b$). *Postfix* puts it after ($ab+$). *Prefix* puts it before ($+ab$). Postfix and prefix need no parentheses or precedence rules — position alone fixes the order. Standard treatment; see Horowitz & Sahni Ch. 3 [1].

Same computation, three spellings: $(a+b)*c = ab+c* = **abc$. Week 3 evaluated the middle spelling; today we produce all three from the first.

4.2.2 Precedence and associativity

Definition (Precedence). *Precedence* ranks operators: it decides $*$ before $+$, so $a+b*c$ means $a+(b*c)$ regardless of left-to-right position.

Definition (Associativity). *Associativity* breaks ties among equal-precedence operators: left-associative — reads $a-b-c$ as $(a-b)-c$, while right-associative $\wedge$ reads a^{b^c} as $a^{(b^c)}$.

The table the deck asks us to memorise once (used by C, Java, and the syllabus alike):

operators	precedence	associativity
()	highest (override)	—
$\wedge$ (power)	high	right: $a^{b^c} = a^{(b^c)}$
$*$ /	medium	left: $a/b*c = (a/b)*c$
$+$ -	low	left: $a-b-c = (a-b)-c$

Example 4.2.1 (Same operands, different spellings). $a+b*c$ means $a+(b*c)$: postfix $abc*+$. The $*$ binds first by precedence, so b and c combine before a ever meets the $+$. $(a+b)*c$ means $(a+b)*c$: postfix $ab+c*$. The brackets force a and b together first, and the $*$ waits until that sum exists. The brackets move no operand — they only redirect which operator waits on the stack. That redirection is the whole conversion algorithm of the next section. Trace both with the Week 3 evaluator to confirm: same stack code, different strings in.

Example 4.2.2 (Prediction check: which postfix matches $a*b+c$?). Of $A: abc*+$, $B: ab*c+$, $C: ab+*c$, the answer is B — multiplication first, by precedence. A evaluates as $a*(b+c)$ (addition first, since it is written last — wrong with no brackets present). C implies brackets around $b+c$ that were never written. Precedence decides this before any stack is even mentioned: $*$ outranks $+$, so a and b combine first ($ab*$), and the $+$ meets that product next ($ab*c+$).

4.3 Infix to Postfix with One Stack

Read alongside: Karumanchi, *Data Structures and Algorithms Made Easy*, Ch. 4, pp.163–204 (PDF) — the stack operations and the conversion algorithm.

The algorithm (Karumanchi Ch. 4, pp.163–204 PDF) [2] scans once, left to right, with one operator stack. Operands never touch the stack; only operators wait there:

```
for each token t in the infix string, left to right:
    operand: append to output
    '(': push it
    ')': pop to output until '(' (discard both parens)
    operator op: while top is an operator of >= precedence
        (strictly > if op is right-associative): pop to output
        then push op
at the end: pop everything to output
```

Read the operator rule carefully, because both traps in this section live in it. A left-associative op flushes every waiting operator of equal *or* higher precedence ($\geq$): $a-b-c$ must pop the first $-$ before pushing the second, or the order comes out right-nested. A right-associative op such as $\wedge$ flushes only strictly higher precedence ($>$): a second $\wedge$ must *not* pop the first, or a^{b^c} comes out as $(a^b)^c$.

Example 4.3.1 (Full trace: $a+b*c-(d+e/f)$). Walk token by token, maintaining the output string and the stack after each step. Start with empty output and an empty stack. Token a is an operand: append, output a , stack empty. Token $+$ is an operator with an empty stack above nothing: push, output a , stack $+$. Token b : append, output $a\ b$, stack $+$. Token $*$: the top $+$ has lower precedence, so pop nothing and push; output $a\ b$, stack $+\ *$. Token c : append, output $a\ b\ c$, stack $+\ *$. Token $-$: left-associative with the lowest precedence, so it flushes everything waiting — pop $*$ then $+$ to the output, then push $-$; output $a\ b\ c\ *\ +$, stack $-$. Token $($: push as a stop-marker; output unchanged, stack $-\ ($. Token d : append, output $a\ b\ c\ *\ +\ d$, stack $-\ ($. Token $+$: the top is $($, which is not an operator, so pop nothing and push; output $a\ b\ c\ *\ +\ d$, stack $-\ (+$. Token e : append, output $a\ b\ c\ *\ +\ d\ e$, stack $-\ (+$. Token $/$: top $+$ is lower, so push; output $a\ b\ c\ *\ +\ d\ e$, stack $-\ (+ /$. Token f : append, output $a\ b\ c\ *\ +\ d\ e\ f$, stack $-\ (+ /$. Token $)$: pop to output until $($ — $/$ then $+$ — and discard both parens; output $a\ b\ c\ *\ +\ d\ e\ f\ /\ +$, stack $-$. End of string: pop everything; output $a\ b\ c\ *\ +\ d\ e\ f\ /\ +\ -$, stack empty. Watch the $-$: it flushes $+$ first, since both outrank or tie it left-associatively.

Three traps, then the price. **Power is right-associative:** a^{b^c} is $a^{(b^c)}$, so a second $\wedge$ must *not* pop the first — pop only on strictly greater precedence. **Parentheses are not output:** $($ marks “stop here” on the stack; $)$ flushes to it and both parens are discarded, never appended. **Subtraction order:** $a-b$ pops as “pop b first, a second” at evaluation — the Week 3 rule, unchanged, so `apply(-, a, b)` keeps non-commutative operators correct. The cost, in Week 2’s shape: name the operation (per-token push/pop), give the bound (one pass, each token pushed and popped at most once: $O(n)$ time worst case, $O(n)$ auxiliary space worst case for n tokens), state the case (worst case).

Example 4.3.2 (Prediction check: convert $(a+b)*c-d$). Of $A: ab+cd-*$, $B: ab+c*d-$, $C: abc*+d-$, the answer is B — brackets first, then $*$, then $-$. Trace it: $($ pushes as a marker; a appends; $+$ pushes above the marker; b appends; $)$ flushes exactly one $+$ and both parens vanish, leaving $ab+$ with an empty stack. $*$ pushes; c appends; $-$, left-associative and lower than $*$, pops the $*$ first, giving $ab+c*$, then pushes; d appends; end-of-string pops the $-$: $ab+c*d-$. A binds $-$ tighter than $*$ (it never was); C lets precedence beat the brackets (brackets always win).

4.4 The Prefix Route and Back Again

Read alongside: Karumanchi, Ch. 4, pp.163–204 (PDF) — the same stack chapter, reversal method and right-to-left scan; expression-tree connection previews Week 11.

4.4.1 Infix to prefix by reversal

The method reuses the postfix algorithm instead of inventing a second one:

1. Reverse the infix string; swap '(' with ')'.
2. Run the postfix algorithm, treating all operators as usual BUT popping only on strictly greater precedence (mirror rule).
3. Reverse the result.

Why it works: prefix of E is the mirror of postfix of E reversed — one algorithm serves both directions. Two details carry the correctness. The paren swap is the step everyone forgets; without it every bracket binds backwards, because reversal turns each opener into a closer positionally. And step 2's mirror rule (pop only on strictly greater precedence) compensates for the reversal of associativity: scanning backwards flips which equal-precedence operator should wait, so the tie-break must flip with it.

Example 4.4.1 (Worked conversion: $a*(b+c)-d$ to prefix). *Step 1, reverse and swap parens: $d-(c+b)*a$. Check mechanically: the original left to right is $a, *, (, b, +, c,), -, d$; reversed it is $d, -,), c, +, b, (, *, a$; swapping each paren gives $d, -, (, c, +, b,), *, a$, i.e. $d-(c+b)*a$. Step 2, postfix of the reversed string: $d\ c\ b\ +\ a\ *\ -$ (the $)$ flushes the $+$ exactly as before — the bracketed sum still converts as a unit). Step 3, reverse back: $-*a+bcd$. Sanity check, reading $-*a+bcd$ outside-in: subtract d from $\times(a, +(b, c))$ — that is $a \times (b+c) - d$. Correct. Fresh instance, same method as the previous section — no new machinery.*

4.4.2 Prefix to postfix with one stack

Scan right to left, stacking strings rather than values:

```
for each token t from right to left:
    if t is an operand: push(t as a string)
    if t is an operator op:
        a = pop(); b = pop()
        push(a + " " + b + " " + op)
at the end: the stack holds the postfix string
```

Direction matters: right-to-left meets operands before the operator that combines them, so when an operator arrives both its operand-strings are already on top. Note the pop order: a is popped first (it was pushed later, hence sits higher, hence is the left operand in the right-to-left scan) and b second; pushing $a\ b\ op$ keeps non-commutative operators correct.

Example 4.4.2 (Trace $**ab-cd$ to postfix). *Scan right to left. Token d : operand, push; stack holds d . Token c : push; stack holds d, c (top rightmost). Token $-$: operator; pop $a = c$, pop $b = d$, push $cd-$; stack holds $cd-$. Token b : push; stack holds $cd-, b$. Token a : push; stack holds $cd-, b, a$. Token $+$: operator; pop $a = a$, pop $b = b$, push $ab+$; stack holds $cd-, ab+$. Token $*$: operator; pop $a = ab+$, pop $b = cd-$, push $ab+cd-*$; stack holds the single string $ab+cd-*$. Done: prefix $**ab-cd$ is postfix $ab+cd-*$.*

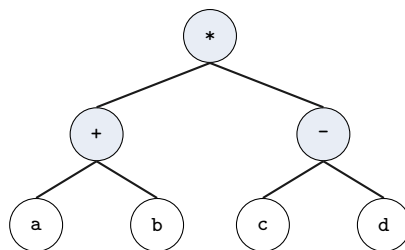


Figure 4.2: Expression tree for $*+ab-cd$: post-order reads $ab+cd-*$, pre-order reads $*+ab-cd$. Full story in Week 11.

Figure 4.2 files one picture away for Week 11: operators are branch nodes, operands are leaves; postfix is post-order, prefix is pre-order. For $*+ab-cd$, the root $*$ has children $+$ and $-$, and the four leaves a , b , c , d hang in order underneath.

Example 4.4.3 (Prediction check: prefix $-+ab+cd$ to postfix). *Of $A: ab+$, $B: cd+$, $C: ab+cd+-$, the first operator resolved is $B - cd+$. Scanning right-to-left meets d , then c , both operands, and then the rightmost $+$ with both its operands waiting — so d and c combine under that $+$ before anything else happens. A ($ab+$, the leftmost operator) resolves only after the scan reaches further left; C (the whole string at once) mistakes “operators wait for both operands” for “everything waits” — in fact each operator fires the moment its own two operands are on top.*

All three conversions cost one pass with each token pushed and popped at most once: $O(n)$ time worst case, $O(n)$ auxiliary space worst case, for n tokens.

4.5 Recursion and the Runtime Stack

Read alongside: Karumanchi, Ch. 2, pp.62–73 (PDF) — recursion, the call stack, recursion versus iteration; Wirth, *Algorithms and Data Structures*, Ch. 3, pp.87–108 — recursive algorithms, when recursion helps and when it does not.

Definition (Recursive function). *A recursive function solves a smaller instance of its own problem: one **base case** answered directly, one **recursive case** that calls itself on strictly smaller input [2, 3].*

The deck’s canonical instance:

```

int fact(int n) {
    if (n <= 1) return 1;          /* base case */
    return n * fact(n - 1);        /* smaller instance */
}

```

The **base case** ($n \leq 1$, answered directly as 1) stops the descent; the **recursive case** ($n * \text{fact}(n-1)$) shrinks the input by exactly one and trusts the smaller call’s answer. Both halves are load-bearing. No base case: infinite descent, the stack filling until the program crashes. Input not shrinking: the same fate by a different road — each call re-issues the same size and the chain never bottoms out.

Iteration keeps one frame and updates variables — constant stack space, always explicit about progress. **Recursion** spends one frame per level — the call chain *is* the progress record, at $O(d)$ auxiliary space for depth d . Prefer recursion when the problem is self-similar (Hanoi, tree walks); prefer iteration when depth is large and the loop is simple (Wirth Ch. 3: when recursion does *not* help). Same power, different ledger: recursion bills the runtime stack.

Definition (Frame (activation record)). A *frame* holds one call's locals and its return address. Each call pushes a frame; each return pops it. Week 1's orientation holds: the stack grows down, the heap grows up.

Figure 4.3 shows four frames alive at once during `hanoi(3, S, D, A):main` with `n = 3` at the bottom, then `hanoi(3, S, D, A)`, then `hanoi(2, S, A, D)`, with `hanoi(1, S, D, A)` running on top. The growth arrow points downward and the `top` label points at the running frame. Depth d means d frames alive — Hanoi's depth is only n , but, as the next section prices, its calls number 2^n .

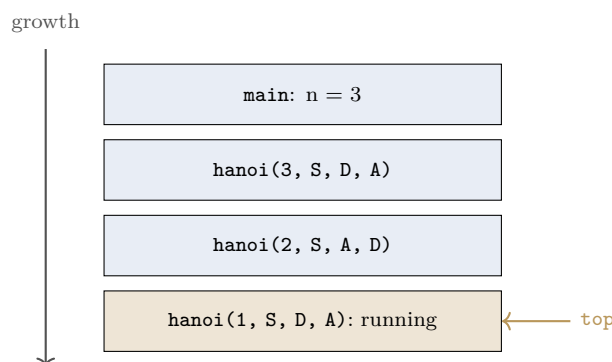


Figure 4.3: Runtime-stack frames during Hanoi with $n = 3$: depth 4 alive, `top` at the running call, growth downward.

4.6 Tower of Hanoi: Self-Similarity, Priced

Read alongside: Karumanchi, Ch. 2, pp.62–73 (PDF) — recurrence setup; Wirth, Ch. 3 — recursive structure; Week 2 vocabulary for naming the operation, giving the bound, and stating the case.

Move the whole tower $S \rightarrow D$; move one disk at a time; never place a larger disk on a smaller one. Figure 4.4 shows the start: three disks stacked on the source peg S , auxiliary peg A and destination peg D empty, the base line beneath and the peg labels below it. Three disks need 7 moves; n disks need $2^n - 1$, proved below.

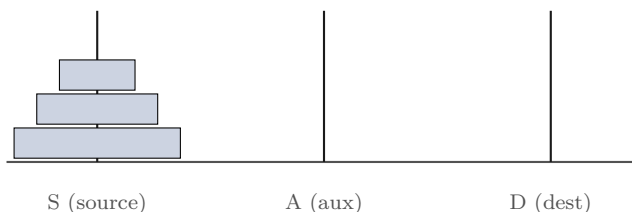


Figure 4.4: Tower of Hanoi, initial position with $n = 3$: the whole tower rests on S ; A and D stand empty.

4.6.1 The recursive idea

```
hanoi(n, src, dst, aux):
    if n == 1: move disk src -> dst
```

```

else:
    hanoi(n-1, src, aux, dst)    /* park n-1 out of the way */
    move disk src -> dst        /* free the largest disk */
    hanoi(n-1, aux, dst, src)    /* stack the n-1 on top */

```

The base case moves one disk. The recursive case reduces n to $n-1$ twice — self-similarity made visible: park the top $n-1$ disks on the spare peg, move the freed largest disk to its destination, then restack the $n-1$ on top of it. Each “move disk” line is one counted step for the analysis next.

Example 4.6.1 (Trace: three disks, seven moves). *Number the disks smallest, middle, largest, all starting on S. Move 1, smallest $S \rightarrow D$: the small disk must vacate so the middle can move. Move 2, middle $S \rightarrow A$: the middle parks on the spare peg. Move 3, smallest $D \rightarrow A$: the small disk restacks on the middle, freeing both S and D. Move 4, largest $S \rightarrow D$: the freed largest disk takes its destination — the midpoint of the whole solution. Move 5, smallest $A \rightarrow S$: the small disk vacates so the middle can move again. Move 6, middle $A \rightarrow D$: the middle restacks on the largest. Move 7, smallest $S \rightarrow D$: the small disk caps the tower. Moves 1–3 park two disks on A; move 4 frees the largest; moves 5–7 restack on D — the algorithm above, executed line by line.*

4.6.2 Pricing Hanoi: $T(n) = 2T(n-1) + 1$

Count disk moves, in Week 2’s vocabulary. $T(1) = 1$; for $n > 1$, $T(n) = 2T(n-1) + 1$ (two sub-towers plus the largest disk’s move). Unfold step by step. One level: $T(n) = 2T(n-1) + 1$. Two levels, substituting $T(n-1) = 2T(n-2) + 1$: $T(n) = 2(2T(n-2) + 1) + 1 = 4T(n-2) + 3$. Three levels: $T(n) = 8T(n-3) + 7$. The pattern after k levels is $T(n) = 2^k T(n-k) + (2^k - 1)$ — each level doubles the pending subproblems and the accumulated +1s form $1 + 2 + 4 + \dots + 2^{k-1} = 2^k - 1$. At $k = n-1$ the recursion bottoms out: $T(n) = 2^{n-1} \cdot 1 + (2^{n-1} - 1) = 2^n - 1$. Hence $T(n) = 2^n - 1 = \Theta(2^n)$ — worst, best, and average coincide, because only one legal move sequence exists (at every non-trivial position exactly one move makes progress without undoing prior work). **Doubling the disks squares the work only in legend — it multiplies the moves by roughly two per added disk, and $2^{64} - 1$ outlives us all.**

4.6.3 What recursion bills you

Three failure modes, each a way of misspending the runtime stack. **Depth overflow**: $n = 100,000$ recursive frames burst the runtime stack — the $O(d)$ ledger comes due, so iterate instead. **Repeated subproblems**: naive recursive Fibonacci recomputes the same values exponentially often (each call re-solves what its siblings already solved) — iteration or memoisation wins. **Missing base case**: the stack fills until the program crashes — the use-after-free of control flow, a frame leak with no return in sight. The habit, in Week 2’s shape: name the operation (moves, frames), give the bound ($\Theta(2^n)$ moves, $O(n)$ frames), state the case.

Example 4.6.2 (Prediction check: Hanoi with $n = 4$). *Of A: 8 moves, 4 frames; B: 15 moves, 4 frames; C: 16 moves, 16 frames — the answer is B. Moves: $2^4 - 1 = 16 - 1 = 15$ (A doubles exactly instead of doubling-minus-one; C confuses 2^n with the move count). Frames: depth never exceeds n , so at most 4 frames are ever alive at once — C mistakes total calls for simultaneous depth. Depth is the longest chain alive; moves count every call ever made.*

4.7 Mistakes, Summary, and What Comes Next

Three mistakes to retire now. **Popping ^ on a tie** — right-associative means strictly-greater pops only; a^{b^c} is not $(a^b)^c$. **Forgetting the paren swap on reversal** — infix-to-prefix without swapping

binds every bracket backwards. **No base case, or input that never shrinks** — Hanoi with $n-1$ missing recurses until the stack bursts. The habit behind all three: trace the stack (operators, frames) line by line — never from memory of the shape.

In summary: **Notations** — infix for humans; postfix/prefix need no parens. **Conversions** — infix to postfix (one left-to-right stack pass); infix to prefix (reverse, swap, postfix, reverse); prefix to postfix (right-to-left stack of strings) — all $O(n)$ time worst case, $O(n)$ auxiliary space worst case. **Recursion** — base plus shrinking self-call; frames on the runtime stack, $O(d)$ space for depth d . **Hanoi** — $T(n) = 2T(n-1) + 1 = 2^n - 1 = \Theta(2^n)$ moves; depth only n . Next week: queues — what changes when the first element out is the first one in — circular buffers, dequeues, and priority.

Exercises

Answer from the algorithms, then check in the lab: both infix conversions are Lab P3, prefix to postfix is Lab P4.

1. Postfix of $(a-b)*(c+d)$? Run the infix-to-postfix algorithm token by token and give the output and stack after each step.
2. Prefix of $a+b*c$? Give it via the reverse–swap–postfix–reverse route, showing the reversed string and its postfix before the final reversal.
3. Moves and max live frames for Hanoi $n = 5$? State both, naming the operation and the case for each bound.
4. Convert $a+b*(c-d)/e$ (this week’s running example) to postfix with a full token trace. At which token does the first pop-to-output happen, and why?
5. Convert prefix $++ab*cd$ to postfix by the right-to-left scan, showing the stack after every token. (Genuinely new instance reusing the deck’s prefix-to-postfix rule.)

Solutions

1. **Postfix $ab-cd+*$.** Trace: (pushes a marker; a appends (a); - pushes above the marker; b appends (a b);) flushes the - and discards the parens (a b -); * pushes; (pushes a second marker; c appends; + pushes above the marker; d appends;) flushes the + (a b - c d +); end of string pops the *: $ab-cd+*$. (The deck’s own “Before You Go” answer, re-derived step by step.)
2. **Prefix $+a*bc$.** Reverse $a+b*c$ to $c*b+a$; postfix of the reversed string under the mirror rule is $cb*a+$; reverse back to $+a*bc$. Check outside-in: add a to $\times(b, c)$ — that is $a + b \times c$. Correct. (The deck’s own “Before You Go” answer.)
3. **31 moves, 5 frames.** Moves: $T(5) = 2^5 - 1 = 31$ disk moves (worst = best = average, one legal sequence). Frames: depth at most $n = 5$ live frames, $O(n)$ auxiliary space worst case. (The deck’s own “Before You Go” answer.)
4. **$abcd-*e/+$.** Trace: a appends; + pushes; b appends; * pushes above + (higher precedence, no pop); (pushes a marker; c appends; - pushes above the marker; d appends;) flushes - and discards the parens; / meets top * of equal precedence, left-associative, so it pops * first, then pushes /; e appends; end pops / then +. The first pop-to-output happens at /: it is the first operator that meets a waiting operator of equal-or-higher precedence (*), forcing the flush.
5. **$ab*cd+*$.** Right to left: d pushes (d); c pushes (d, c with c on top); * pops $a = c$, $b = d$, pushes $cd*$ (stack: $cd*$); b pushes (stack: $cd*$, b); a pushes (stack: $cd*$, b, a); * pops $a = a$, $b = b$, pushes $ab*$ (stack: $cd*$, $ab*$); + pops $a = ab*$, $b = cd*$, pushes $ab*cd+*$. Outside-in check: add $\times(a, b)$ to $\times(c, d)$ — the correct sum of products.

References

- [1] Ellis Horowitz and Sartaj Sahni. *Fundamentals of Data Structures*. Galgotia Booksources, 2nd edition, 2008.
- [2] Narasimha Karumanchi. *Data Structures And Algorithms Made Easy*. CareerMonk Publications, 2017. This calibre-generated edition carries no printed folio numbers; page numbers cited in CS301 material are PDF page indices. NOT a source for Week 1’s C-specific pointer/malloc content — see this course’s supporting-books index for the coverage check.
- [3] Niklaus Wirth. *Algorithms and Data Structures*. Author (freely distributed), 2004. Oberon version, August 2004; revision of the 1985 Prentice-Hall edition. Page numbers cited in CS301 material refer to this Oberon PDF, not to the 1985 print edition.